

PROFESSIONAL LEARNING EDITION

JavaScript & React Study Guide

Beginner roadmap rebuilt into a cleaner, more professional PDF with clear bullet markers, clickable navigation, and example code for every core learning module.

Use it as a study checklist, a revision guide, and a project-planning reference while you build your first React apps.

What is improved

- Stronger visual hierarchy and spacing
- Clickable table of contents and back links
- Example code in every main topic
- Visible checklist-style bullet markers

How to use this guide

- Read modules in order the first time
- Build one tiny example after each module
- Check off the practice tasks as you go
- Return to the appendices for roadmap planning

CONTENTS

Clickable Table of Contents

Tip: every section title below links to its module, and every module has a back-to-contents link for faster revision.

- [JavaScript Fundamentals](#)
- [TypeScript Basics](#)
- [React Basics](#)
- [Debugging](#)
- [Styling Components](#)
- [Component Deep Dive](#)
- [HTTP Requests & API Handling](#)
- [Routing](#)
- [Forms & Validation](#)
- [Redux](#)
- [Authentication](#)
- [Testing Introduction](#)
- [Deployment](#)
- [Advanced Topics](#)

FOCUS FIRST

JavaScript, React basics, hooks, forms, API calls, routing.

BUILD WHILE LEARNING

Counter app, todo app, weather app, blog app, auth dashboard.

USE APPENDICES FOR

Weekly roadmap, project ideas, common mistakes, final advice.

JavaScript Fundamentals

React becomes much easier once variables, functions, arrays, objects, and references feel natural.

Core ideas

- Prefer **const** by default and use **let** only when reassignment is required.
- Use arrow functions for callbacks and compact reusable logic.
- Learn **destructuring**, **spread**, and **rest** because they appear everywhere in React state updates.
- Practice **map**, **filter**, and **reduce** so rendering lists feels easy.
- Understand that arrays and objects are copied by reference unless you create a new copy.

Common mistake: mutating an object directly and expecting React to notice a change.

Practice checklist

- ☐ Rewrite a normal function as an arrow function.
- ☐ Use destructuring in a function parameter.
- ☐ Copy an array with spread before adding an item.
- ☐ Transform an array with **map**.

EXAMPLE CODE

```
const users = [
  { id: 1, name: 'Khwaab', score: 82 },
  { id: 2, name: 'Sam', score: 95 }
];

const addBonus = (list, bonus) =>
  list.map(({ score, ...user }) => ({
    ...user,
    score: score + bonus
  }));

const updatedUsers = addBonus(users, 5);
// [{ id: 1, name: 'Khwaab', score: 87 }, ...]
```

TypeScript Basics

TypeScript helps you catch mistakes earlier and makes large React apps easier to refactor safely.

What to learn

- Primitive types, arrays, objects, and function return types.
- **type** and **interface** for describing data shapes.
- Union types for values that can be one of several options.
- Typed component props for safer React components.

Watch out for

- ☐ Do not use **any** everywhere.
- ☐ Type function parameters and return values.
- ☐ Keep prop names and types consistent.

EXAMPLE CODE

```
type User = {
  id: number;
  name: string;
  role: 'admin' | 'member';
};

type UserBadgeProps = {
  user: User;
};

function UserBadge({ user }: UserBadgeProps) {
  return <span>{user.name} ({user.role})</span>;
}
```

React Basics

This is where JavaScript turns into UI: components, JSX, props, state, events, and small reusable pieces.

Essential ideas

- Modern projects are commonly created with Vite.
- Components return JSX, which looks like HTML inside JavaScript.
- **Props** pass data in; **state** stores changing data.
- Event handlers such as **onClick** and **onChange** keep the UI interactive.
- Controlled inputs make forms predictable.

Build this first

- ☐ A greeting component with props.
- ☐ A counter with state and a button click.
- ☐ A text input that updates live on screen.

EXAMPLE CODE

```
import { useState } from 'react';

function WelcomeCard({ name }) {
  const [likes, setLikes] = useState(0);

  return (
    <section className="card">
      <h2>Hello, {name}</h2>
      <p>Likes: {likes}</p>
      <button onClick={() => setLikes(likes + 1)}>Like</button>
    </section>
  );
}
```

Debugging

Debugging is a skill, not a last resort. The goal is to find the smallest place where expected behavior diverges from actual behavior.

Debugging workflow

- Read the full error message, especially the file name and line number.
- Reproduce the issue with the smallest possible input.
- Use **console.log**, browser DevTools, and React Developer Tools.
- Check props, state, and network responses before guessing.

Useful checks

- ☐ Is a value undefined before it loads?
- ☐ Did you mutate state instead of replacing it?
- ☐ Is a component rendering more often than expected?

EXAMPLE CODE

```
function Profile({ user }) {  
  console.log('Profile received:', user);  
  
  if (!user) {  
    return <p>Loading user...</p>;  
  }  
  
  return <h2>{user.name}</h2>;  
}
```

Tip: guarding against missing data first often removes “Cannot read properties of undefined” errors.

Styling Components

Start with a simple styling approach, then choose utility classes or component-scoped styles when the project grows.

Common options

- Plain CSS for small projects and quick experiments.
- CSS Modules for local component styles.
- Tailwind CSS for utility-first styling.
- styled-components for CSS-in-JS projects.

Remember

- ☐ Use conditional classes for active or disabled states.
- ☐ Keep style naming consistent.
- ☐ Do not overcomplicate styling for beginner apps.

EXAMPLE CODE

```
function StatusBadge({ isActive }) {  
  const className = isActive  
    ? 'badge badge--active'  
    : 'badge badge--inactive';  
  
  return <span className={className}>  
    {isActive ? 'Active' : 'Inactive'}  
  </span>;  
}
```


Component Deep Dive

As apps grow, good component structure matters just as much as writing JSX that works.

What to understand

- Split large screens into smaller focused components.
- Prefer functional components and hooks in modern React.
- Use **Fragment** when you need multiple sibling elements without an extra wrapper.
- Use **refs** for DOM access and focus management.
- Use **React.memo** only when repeated renders are actually a problem.

Component habits

- ☐ Keep components small and purpose-specific.
- ☐ Move repeated logic into custom hooks.
- ☐ Name components after what they render.

EXAMPLE CODE

```
import { useEffect, useRef } from 'react';

function SearchInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  return (
    <>
      <label htmlFor="search">Search</label>
      <input id="search" ref={inputRef} />
    </>
  );
}
```

HTTP Requests & API Handling

Fetching data is one of the first places where real apps feel different from toy examples.

Key ideas

- Use **GET** to fetch, **POST** to create, **PATCH/PUT** to update, and **DELETE** to remove.
- Store loading, data, and error state explicitly.
- Transform API data before rendering if the UI needs a different shape.
- Avoid infinite loops in **useEffect** dependencies.
- An axios instance can centralize a base URL and shared headers.

Checklist

- ☐ Show a loading state.
- ☐ Show an error state.
- ☐ Use keys when rendering API lists.
- ☐ Keep fetch logic predictable.

EXAMPLE CODE

```
import { useEffect, useState } from 'react';

function PostsList() {
  const [posts, setPosts] = useState([]);
  const [status, setStatus] = useState('idle');

  useEffect(() => {
    setStatus('loading');

    fetch('/api/posts')
      .then((res) => res.json())
      .then((data) => {
        setPosts(data);
        setStatus('success');
      })
      .catch(() => setStatus('error'));
  }, []);

  if (status === 'loading') return <p>Loading...</p>;
  if (status === 'error') return <p>Could not load posts.</p>;

  return <ul>{posts.map((post) => <li key={post.id}>{post.title}</li>)}</ul>;
}
```

Routing

Routing chooses which component to show for a URL, making your app feel like a multi-page experience without full page reloads.

Topics to cover

- **react-router-dom** for route definitions and navigation.
- **Link** for internal navigation instead of a normal anchor tag.
- Route parameters such as **/products/:id**.
- Protected routes, 404 routes, and lazy-loaded pages.

Mini goals

- ☐ Create Home, About, and Product pages.
- ☐ Read a route parameter with **useParams**.
- ☐ Redirect unauthenticated users.

EXAMPLE CODE

```
import { BrowserRouter, Link, Navigate, Route, Routes } from 'react-router-dom';

function ProtectedRoute({ isLoggedIn, children }) {
  return isLoggedIn ? children : <Navigate to="/login" replace />;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/dashboard">Dashboard</Link>
      </nav>

      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
        <Route
          path="/dashboard"
          element={
            <ProtectedRoute isLoggedIn={true}>
              <h1>Dashboard</h1>
            </ProtectedRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  );
}
```

Forms & Validation

Forms become easier when every input is controlled and validation rules are written clearly.

Key practices

- Store form values in state.
- Update fields with **onChange**.
- Prevent default form refresh on submit.
- Show validation feedback next to the related field.
- Never render raw untrusted HTML into the page.

Validation ideas

- ☐ Required fields
- ☐ Minimum length
- ☐ Email format
- ☐ Password match

EXAMPLE CODE

```
import { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState('');
  const [error, setError] = useState('');

  function handleSubmit(event) {
    event.preventDefault();

    if (!email.includes('@')) {
      setError('Please enter a valid email address. ');
      return;
    }

    setError('');
    console.log('Submit form');
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={(e) => setEmail(e.target.value)} />
      {error && <p>{error}</p>}
      <button type="submit">Login</button>
    </form>
  );
}
```

Redux

Redux is most useful when many parts of the app need the same state and local state starts getting messy.

Understand this flow

- UI dispatches an action.
- A reducer updates state.
- The store holds the latest state.
- Components read values with selectors.
- Redux Toolkit is the modern default choice.

Use Redux when

- ☐ Many distant components need the same data.
- ☐ App state is too large for simple lifting.
- ☐ You want consistent action-driven updates.

EXAMPLE CODE

```
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    addItem(state, action) {
      state.items.push(action.payload);
    }
  }
});

export const { addItem } = cartSlice.actions;
export default cartSlice.reducer;
```

Rule of thumb: learn Context first for small shared state and Redux Toolkit for larger apps.

Authentication

Authentication is about identifying the user safely and controlling access to protected areas of the app.

Core concepts

- Login, signup, logout, and protected routes.
- JWT basics and session refresh concepts.
- HTTP-only cookies are often safer than storing sensitive tokens in localStorage.
- Auth guards decide whether a route is visible.

Security notes

- ☐ Prefer server-managed auth flows when possible.
- ☐ Avoid exposing sensitive tokens to JavaScript unless you must.
- ☐ Separate auth state from presentational UI.

EXAMPLE CODE

```
import { Navigate } from 'react-router-dom';

function ProtectedRoute({ user, children }) {
  if (!user) {
    return <Navigate to="/login" replace />;
  }

  return children;
}
```

Testing Introduction

Good tests focus on what the user sees and does, not the private implementation details inside a component.

Cover these states

- Success and failure
- Loading and empty states
- Clicks, typing, and submit flows
- Visible feedback after user actions

Testing habit

- ☐ Render the component.
- ☐ Perform a user action.
- ☐ Assert what changed on screen.

EXAMPLE CODE

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('increments the counter', async () => {
  render(<Counter />);

  await userEvent.click(screen.getByRole('button', { name: /like/i }));

  expect(screen.getByText(/likes: 1/i)).toBeInTheDocument();
});
```

Deployment

Deployment means packaging your app correctly, checking environment settings, and making sure client-side routes still work after refresh.

Before deploy

- Run a production build and fix errors.
- Confirm base paths and asset paths.
- Check API URLs and environment variables.
- Make sure the server falls back to **index.html** for SPA routes.

Final checks

- ☐ App loads after a page refresh.
- ☐ Console has no critical errors.
- ☐ Protected routes still behave correctly.

EXAMPLE CODE

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  base: '/portfolio/'
});
```

Advanced Topics

These topics become more valuable after the fundamentals feel comfortable. Learn them for leverage, not for memorization.

Explore next

- Core hooks such as **useEffect**, **useMemo**, **useCallback**, and **useContext**.
- Custom hooks for reusable stateful logic.
- Context API for shared app-level values.
- Webpack and Babel concepts so tooling feels less mysterious.
- Animations and transitions for UI polish.

Learning advice

- ☐ Do not start with Saga.
- ☐ Use custom hooks when logic repeats.
- ☐ Reach for memoization only when needed.

EXAMPLE CODE

```
import { useState } from 'react';

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  function increment() {
    setCount((current) => current + 1);
  }

  return { count, increment };
}
```

APPENDIX A

What to Learn First: 6-Week Roadmap

This keeps the learning order focused so you are building early instead of waiting until everything feels complete.

Week 1

- ☐ let and const
- ☐ arrow functions
- ☐ imports and exports
- ☐ spread, rest, and destructuring
- ☐ array methods

Week 2

- ☐ TypeScript basics
- ☐ React workflow and JSX
- ☐ props, children, and state
- ☐ events and dynamic rendering

Week 3

- ☐ hooks and effects
- ☐ refs
- ☐ styling
- ☐ forms and validation

Week 4

- ☐ fetch and axios
- ☐ routing and protected routes
- ☐ 404 pages and lazy loading

Week 5

- ☐ Context API
- ☐ Redux Toolkit basics
- ☐ testing basics
- ☐ deployment basics

Week 6 and Beyond

- ☐ memoization patterns
- ☐ advanced debugging
- ☐ authentication patterns
- ☐ animations, webpack, and Babel

APPENDIX B

Best Beginner Projects

Each project below teaches a specific cluster of skills, so use them as checkpoints rather than random app ideas.

Counter App

Learn state, events, and rendering values.

- Increment and decrement buttons
- Disable the minus button at zero

Todo App

Learn lists, keys, local state, and form handling.

- Add, toggle, and delete tasks
- Filter by completed status

Notes App

Learn CRUD operations and structured component state.

- Create and edit notes
- Store note data in arrays of objects

Weather App

Learn API fetching, loading states, and error handling.

- Search by city
- Show loading, success, and error states

Expense Tracker

Learn derived values, filters, and reusable form components.

- Add categories and totals
- Filter by date or category

Blog App / Auth Dashboard

Learn routing, API data, protected pages, and shared state.

- Route to a dynamic detail page
- Protect a dashboard route after login

APPENDIX C

Common Mistakes Beginners Make

These mistakes are normal. The goal is to notice them earlier and build the habit of correcting them quickly.

- Mutating state directly

[Fix pattern](#)

- Forgetting keys when rendering lists
- Using **useEffect** incorrectly and causing loops
- Keeping too much state instead of deriving values
- Using advanced libraries before mastering basics
- Memorizing syntax without building projects

- ☐ Replace state instead of mutating it.
- ☐ Give every rendered list item a stable key.
- ☐ Practice tiny projects after each module.

WRONG VS RIGHT

```
// Wrong
user.name = 'New name';
setUser(user);

// Right
setUser((current) => ({
  ...current,
  name: 'New name'
}));
```

APPENDIX D

Final Advice

You do not need to master everything before building. The fastest path is learning one concept, building one small example, and repeating.

Highest-value path

- JavaScript fundamentals
- React basics
- Hooks
- Forms

Learning loop

- ☐ Learn one concept.
- ☐ Build one tiny example.
- ☐ Break it and debug it.
- ☐ Repeat with the next concept.

- API calls
- Routing
- Context or Redux Toolkit
- Testing and deployment

Keep perspective: legacy topics like class components, HOCs, Radium, and Saga are worth recognizing, but they should not slow down your modern React learning path.

Prepared as a cleaner, example-rich edition of the original study guide.